Отчёт по разработке программы оформления заказа в аптечном пункте

Филимонов А.В.
П-30

1. Цель работы

Разрабатывается консольная программа на языке C# для оформления заказа в аптечном пункте. Пользователь выбирает препараты из ассортимента и указывает их количество. Программа проверяет наличие товаров, рассчитывает стоимость заказа и выводит остатки на складе.

2. Структура программы

Программа состоит из четырёх основных частей:

• Program в пространстве имён Work3 — содержит точку входа Main и организует общую последовательность выполнения и вызовов функций;

• Func в пространстве имён Work3 — выполняет роль основного модуля бизнес-логики, хранит ассортимент и содержит статические методы взаимодействия с пользователем и обработки заказа;

• Product в пространстве имён Work3— описывает модель лекарственного препарата и инкапсулирует его характеристики.

- ProductTypeEnum в пространстве имён Work3 - выполняет роль хранения перечисления и его настройки(соотношение индексов со значением)

Метод Main координирует работу всей системы: вызывает вывод ассортимента на экран, инициализирует процедуру формирования заказа, запускает комплексную проверку наличия и расчёт стоимости, после чего отображает финальные складские остатки.

3. Класс Product и ассортимент товаров

Класс Product описывает сущность лекарственного средства и содержит скрытые (private) поля для безопасного хранения информации:

• name_ (string) — внутреннее наименование препарата;

• price_ (int) — стоимость одной упаковки товара;

• stock_ (int) — доступный остаток на складе.

Инициализация полей выполняется через публичный конструктор Product (string name, int price, int stock). Доступ к скрытым полям из внешних модулей предоставляется строго через методы-геттеры GetName(), GetPrice() и GetStock(). Для уменьшения складского остатка при успешном заказе предусмотрен специализированный метод ReduceStock(int quantity).

Перечисление ProductType в пространстве имён product служит для строгой типизации доступных медикаментов. В программе жёстко задан следующий базовый ассортимент:

Препарат (ProductType)	**Цена за упаковку**	**Остаток на складе**
Анальгин (Analgin = 1)	35 руб.	30 упаковок
Аспирин (Aspirin = 2)	40 руб.	25 упаковок
Йод (Iodine = 3)	120 руб.	20 упаковок
Амоксициллин (Amoxicillin = 4)	650 руб.	12 упаковок
Витамины (Vitamins = 5)	480 руб.	15 упаковок

4. Реализованные функции и методы

Вся логика распределена по статическим методам класса functions:

• **GetProduct(ProductType type) —** приватный метод, возвращающий конкретный объект Product из внутренней коллекции на основе переданного типа препарата с помощью конструкции switch(type). Реализована генерация исключения ArgumentException при передаче неизвестного типа.

• **PrintAssortment() —** выводит в консоль полную таблицу доступных товаров. Индексы позиций формируются автоматически в цикле от 1 до максимального значения перечисления. Информация выводится с использованием методов GetName(), GetPrice() и GetStock().

• **ReadIntInRange(string prompt, int min, int max) —** обеспечивает контролируемое считывание целого числа из консоли. Принимает строку приглашения и границы диапазона. Повторяет запрос при вводе нецелого значения или при выходе за границы диапазона [min; max].

• **ReadNonNegativeInt(string prompt) —** считывает корректное количество упаковок. Защищает программу от ввода некорректного формата текста или отрицательных значений, циклически повторяя запрос до ввода верных данных.

• **ReadOrder() —** интерактивно формирует корзину покупателя. Создаёт целочисленный массив requested, размер которого равен числу элементов в ProductType. Добавляет выбранные объёмы в соответствующие ячейки, автоматически суммируя количества при повторном выборе одной и той же позиции. Ввод завершается при указании номера препарата '0'.

• **TryProcessOrder(int[] requested, out int totalCost, out string missingProductName) —** выполняет двухэтапную обработку заказа. На первом этапе проверяется наличие необходимого объёма по всем позициям. Если хотя бы одного препарата не хватает, метод возвращает false и через out-параметр передаёт имя недостающего товара. Если товаров достаточно, запускается второй этап: вызывается ReduceStock для списания остатков, рассчитывается суммарная стоимость totalCost и метод возвращает true.

• **PrintStocks() —** выводит на экран обновлённую информацию по остаткам упаковок на складе для всех видов продукции в одну строку через запятую.

5. Последовательность выполнения программы

1. Программа запускается и мгновенно выводит начальный ассортимент с актуальными ценами и складскими остатками на экран.

2. Система переходит в режим ожидания ввода от пользователя и запрашивает желаемый номер препарата.

3. Пользователь последовательно вводит номера позиций и требуемое количество упаковок.

a. Для остановки формирования корзины вводится значение 0.

4. Программа передаёт сформированный массив requested в метод комплексной обработки заказа.

5. В случае успеха (запасов хватает) складские остатки уменьшаются, программа выводит итоговую стоимость заказа в рублях.

a. В случае неуспеха (дефицит товара) списание блокируется, а на экран выводится сообщение с точным указанием названия недостающего медикамента.

6. В качестве завершающего шага программа печатает текущие остатки упаковок на складе для контроля корректности данных.

6. Обработка ошибок и валидация данных

Валидация входящей информации реализована на уровне консольного ввода и бизнес-логики.

Методы ReadIntInRange и ReadNonNegativeInt используют встроенную функцию int.TryParse, что полностью исключает падение программы (генерацию необрабатываемых исключений FormatException) при случайном вводе текстовых символов вместо цифр.

Логическая транзакционность метода TryProcessOrder гарантирует целостность базы данных склада: если заказ не может быть выполнен целиком, складские остатки не изменяются ни по одной из позиций.

7. Ожидаемый результат работы программы

Готовая консольная программа представляет собой систему для фармацевтического пункта. Она наглядно отображает номенклатуру товаров, осуществляет циклическую проверку пользовательских параметров, гибко суммирует дублирующиеся позиции в корзине заказа и проводит безопасные транзакции списания. В результате работы минимизируются риски возникновения ошибок из-за человеческого фактора, обеспечивается точный расчёт стоимости покупок и поддерживается актуальная отчётность по остаткам на складе.